

```

In [1]: # Bài 1. Tạo tensor và kiểm tra thuộc tính.
# Yêu cầu:
# 1. Điền ba'ng các giá trị .shape, .dim(), .dtype, .numel() cho a, b, c
# 2. Chạy code kiểm chứng và gia'i thích nếu k'ết qua' khác dự đoán.
# 3. So sánh .shape và .size(). Thử a.size(0) và a.shape[0].
!pip install torch
import torch

a = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
b = torch.zeros(3, 3)
c = torch.randn(2, 4)

# Dự đoán trước khi chạy:
# Tensor | .shape | .dim() | .dtype | .numel()
# a | torch.Size([2,3]) | 2 | torch.float32 | 6
# b | torch.Size([3,3]) | 2 | torch.float32 | 9
# c | torch.Size([2,4]) | 2 | torch.float32 | 8

print("=== Tensor a ===")
print("shape :", a.shape)
print("dim() :", a.dim())
print("dtype :", a.dtype)
print("numel():", a.numel())

print("\n=== Tensor b ===")
print("shape :", b.shape)
print("dim() :", b.dim())
print("dtype :", b.dtype)
print("numel():", b.numel())

print("\n=== Tensor c ===")
print("shape :", c.shape)
print("dim() :", c.dim())
print("dtype :", c.dtype)
print("numel():", c.numel())

# So sánh .shape vs .size()
print("\n=== So sánh .shape và .size() ===")
print("a.shape :", a.shape) # tra' về torch.Size
print("a.size() :", a.size()) # cũng tra' về torch.Size (tương đư
print("a.size(0) :", a.size(0)) # số' hàng
print("a.shape[0] :", a.shape[0]) # số' hàng (cùng k'ết qua')
# K'ết luận: .shape là thuộc tính, .size() là phương thức – k'ết qua' như nh

```

Requirement already satisfied: torch in ./venv/lib/python3.12/site-packages (2.11.0)

Requirement already satisfied: filelock in ./venv/lib/python3.12/site-packages (from torch) (3.25.2)

Requirement already satisfied: typing-extensions>=4.10.0 in ./venv/lib/python3.12/site-packages (from torch) (4.15.0)

Requirement already satisfied: setuptools<82 in ./venv/lib/python3.12/site-packages (from torch) (81.0.0)

Requirement already satisfied: sympy>=1.13.3 in ./venv/lib/python3.12/site-packages (from torch) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in ./venv/lib/python3.12/site-packages (from torch) (3.6.1)

Requirement already satisfied: jinja2 in ./venv/lib/python3.12/site-packages (from torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in ./venv/lib/python3.12/site-packages (from torch) (2026.2.0)

Requirement already satisfied: cuda-toolkit==13.0.2 in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.2)

Requirement already satisfied: cuda-bindings<14,>=13.0.3 in ./venv/lib/python3.12/site-packages (from torch) (13.2.0)

Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in ./venv/lib/python3.12/site-packages (from torch) (9.19.0.56)

Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in ./venv/lib/python3.12/site-packages (from torch) (0.8.0)

Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in ./venv/lib/python3.12/site-packages (from torch) (2.28.9)

Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in ./venv/lib/python3.12/site-packages (from torch) (3.4.5)

Requirement already satisfied: triton==3.6.0 in ./venv/lib/python3.12/site-packages (from torch) (3.6.0)

Requirement already satisfied: nvidia-cublas==13.1.0.3.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.1.0.3)

Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.96)

Requirement already satisfied: nvidia-cufft==12.0.0.61.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (12.0.0.61)

Requirement already satisfied: nvidia-cufile==1.15.1.6.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (1.15.1.6)

Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.85)

Requirement already satisfied: nvidia-curand==10.4.0.35.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (10.4.0.35)

```

Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (12.0.4.66)
Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (12.6.3.3)
Requirement already satisfied: nvidia-cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in ./venv/lib/python3.12/site-packages (from cuda-toolkit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparsenvjitlink,nvrtc,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: cuda-pathfinder~1.1 in ./venv/lib/python3.12/site-packages (from cuda-bindings<14,>=13.0.3->torch) (1.5.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in ./venv/lib/python3.12/site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in ./venv/lib/python3.12/site-packages (from jinja2->torch) (3.0.3)
=== Tensor a ===
shape : torch.Size([2, 3])
dim()  : 2
dtype  : torch.float32
numel(): 6

=== Tensor b ===
shape : torch.Size([3, 3])
dim()  : 2
dtype  : torch.float32
numel(): 9

=== Tensor c ===
shape : torch.Size([2, 4])
dim()  : 2
dtype  : torch.float32
numel(): 8

=== So sánh .shape và .size() ===
a.shape      : torch.Size([2, 3])
a.size()     : torch.Size([2, 3])
a.size(0)    : 2
a.shape[0]   : 2

```

```

In [2]: # Bài 2. Tạo tensor theo yêu cầu (không dùng giá trị cứng).
        # Viết code thực hiện (mỗi yêu cầu chỉ dùng một lời gọi hàm):
        # 1. Ma trận đơn vị 5x5.
        # 2. Vector 8 số cách đều từ 0 đến 1.
        # 3. Tensor shape (3,4) toàn giá trị -1.
        # 4. Tensor shape (2,3,4) chứa các số từ 0 đến 23 (arange + reshape).

```

```
# 5. Tensor shape (4,4) đường chéo chính là [1,2,3,4], còn lại = 0.
```

```
import torch
```

```
# 1. Ma trận đơn vị 5x5
```

```
t1 = torch.eye(5)
```

```
print("1. Ma trận đơn vị 5x5:\n", t1)
```

```
# 2. Vector 8 số cách đều từ 0 đến 1
```

```
t2 = torch.linspace(0, 1, 8)
```

```
print("\n2. Vector linspace(0,1,8):\n", t2)
```

```
# 3. Tensor shape (3,4) toàn giá trị -1
```

```
t3 = torch.full((3, 4), -1.0)
```

```
print("\n3. Tensor full((3,4), -1):\n", t3)
```

```
# 4. Tensor shape (2,3,4) chứa 0..23
```

```
t4 = torch.arange(24).reshape(2, 3, 4)
```

```
print("\n4. arange(24).reshape(2,3,4):\n", t4)
```

```
# 5. Tensor 4x4 với đường chéo [1,2,3,4]
```

```
t5 = torch.diag(torch.tensor([1, 2, 3, 4]))
```

```
print("\n5. Tensor đường chéo [1,2,3,4]:\n", t5)
```

1. Ma trận đơn vị 5x5:

```
tensor([[1., 0., 0., 0., 0.],
        [0., 1., 0., 0., 0.],
        [0., 0., 1., 0., 0.],
        [0., 0., 0., 1., 0.],
        [0., 0., 0., 0., 1.]])
```

2. Vector linspace(0,1,8):

```
tensor([0.0000, 0.1429, 0.2857, 0.4286, 0.5714, 0.7143, 0.8571, 1.0000])
```

3. Tensor full((3,4), -1):

```
tensor([[-1., -1., -1., -1.],
        [-1., -1., -1., -1.],
        [-1., -1., -1., -1.]])
```

4. arange(24).reshape(2,3,4):

```
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],
        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]])
```

5. Tensor đường chéo [1,2,3,4]:

```
tensor([[1, 0, 0, 0],
        [0, 2, 0, 0],
        [0, 0, 3, 0],
        [0, 0, 0, 4]])
```

In [3]: # Bài 3. Indexing và Slicing.

```
# Cho x = torch.arange(1, 17).reshape(4, 4)
```

```

# Dự đoán và kiểm chứng: x[1,2], x[:,3], x[2:4,1:3],
# x[x%3==0], x[-1,-1], x[[0,2],:]

import torch

x = torch.arange(1, 17).reshape(4, 4)
print("x:\n", x)

# x[1, 2] → phần tử hàng 1, cột 2 (0-indexed) = 7
print("\nx[1, 2]      =", x[1, 2])

# x[:, 3] → toàn bộ cột 3 = [4, 8, 12, 16]
print("x[:, 3]       =", x[:, 3])

# x[2:4, 1:3] → hàng 2-3, cột 1-2 = [[10,11],[14,15]]
print("x[2:4, 1:3]    =\n", x[2:4, 1:3])

# x[x % 3 == 0] → các phần tử chia hết cho 3 = [3,6,9,12,15]
print("x[x%3==0]      =", x[x % 3 == 0])

# x[-1, -1] → phần tử cuối cùng = 16
print("x[-1, -1]      =", x[-1, -1])

# x[[0,2], :] → hàng 0 và hàng 2
print("x[[0, 2], :]   =\n", x[[0, 2], :])

```

```

x:
tensor([[ 1,  2,  3,  4],
        [ 5,  6,  7,  8],
        [ 9, 10, 11, 12],
        [13, 14, 15, 16]])

x[1, 2]      = tensor(7)
x[:, 3]      = tensor([ 4,  8, 12, 16])
x[2:4, 1:3]  =
  tensor([[10, 11],
          [14, 15]])
x[x%3==0]    = tensor([ 3,  6,  9, 12, 15])
x[-1, -1]    = tensor(16)
x[[0, 2], :] =
  tensor([[ 1,  2,  3,  4],
          [ 9, 10, 11, 12]])

```

```

In [4]: # Bài 4. Reshape và Views.
# Cho x = torch.arange(12).
# 1. Phân biệt view(3,4) và reshape(3,4).
# 2. Reshape x về (2,2,3).
# 3. Gợi ý thích tham số -1 trong x.view(-1, 2).

import torch

x = torch.arange(12)
print("x:", x)

# 1. view vs reshape
# view() yêu cầu tensor phải contiguous trong bộ nhớ, nếu không sẽ lỗi.

```

```

# reshape() linh hoạt hơn: tự chọn dùng view hoặc copy khi cần.
xv = x.view(3, 4)
xr = x.reshape(3, 4)
print("\nview(3,4):\n", xv)
print("reshape(3,4):\n", xr)
print("Cùng bộ nhớ với view:", xv.storage().data_ptr() == x.storage().data_ptr())

# 2. Reshape về (2,2,3)
x3d = x.reshape(2, 2, 3)
print("\nreshape(2,2,3):\n", x3d)

# 3. Tham số -1 trong view(-1, 2)
# -1 nghĩa là PyTorch tự tính chiều đó dựa trên tổng số phần tử.
# x có 12 phần tử, view(-1, 2) → shape = (6, 2)
xauto = x.view(-1, 2)
print("\nview(-1, 2) → shape:", xauto.shape)
print(xauto)

```

```
x: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
view(3,4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
reshape(3,4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

```
Cùng bộ nhớ với view: True
```

```
reshape(2,2,3):
tensor([[[ 0,  1,  2],
         [ 3,  4,  5]],

        [[ 6,  7,  8],
         [ 9, 10, 11]]])
```

```
view(-1, 2) → shape: torch.Size([6, 2])
tensor([[ 0,  1],
        [ 2,  3],
        [ 4,  5],
        [ 6,  7],
        [ 8,  9],
        [10, 11]])
```

```

/tmp/ipykernel_578971/3976547278.py:19: UserWarning: TypedStorage is deprecated. It will be removed in the future and UntypedStorage will be the only storage class. This should only matter to you if you are using storages directly. To access UntypedStorage directly, use tensor.untyped_storage() instead of tensor.storage()
  print("Cùng bộ nhớ với view:", xv.storage().data_ptr() == x.storage().data_ptr())

```

```

In [5]: # Bài 5. Phép Toán Ma Trận.
# Cho A = torch.randn(3,4) và B = torch.randn(4,5).
# 1. Tính tích ma trận A và B (matmul hoặc @).
# 2. Giả'i thích tại sao A * B báo lỗi. Phân biệt element-wise và dot product.

```

```
# 3. Broadcasting với vector b shape (4,1) để A * b không lỗi.

import torch

A = torch.randn(3, 4)
B = torch.randn(4, 5)

# 1. Tích ma trận
C = torch.matmul(A, B) # hoặc: C = A @ B
print("A @ B shape:", C.shape) # (3, 5)
print("A @ B:\n", C)

# 2. A * B báo lỗi vì A=(3,4) và B=(4,5) khác shape → không broadcast được
# Element-wise (*) yêu cầu cùng shape (hoặc có thể broadcast).
# Dot product (matmul/@) yêu cầu chiều trong khớp: (3,4)·(4,5) → (3,5).
try:
    _ = A * B
except RuntimeError as e:
    print("\nLỗi A * B:", e)

# 3. Broadcasting: b shape (4,1) → A * b hợp lệ vì (3,4) * (4,1)? vẫn lỗi.
# Để nhân element-wise từng cột của A với b ta cần b shape (1,4) hoặc (4,1)
# Để yêu cầu b=(4,1), nên cần transpose A hoặc dùng b=(1,4).
# b shape (3,1):
b_row = torch.randn(3, 1) # (3,1) broadcast với (3,4)
print("\nA * b_row shape (3,1):", (A * b_row).shape)

# Hoặc b=(4,) broadcast với mỗi hàng của A
b_col = torch.randn(4)
print("A * b_col shape (4,) :", (A * b_col).shape)
```

A @ B shape: torch.Size([3, 5])

A @ B:

```
tensor([[ 5.4954,  0.5293,  1.2848,  2.2960, -0.9903],
        [ 3.5328,  3.3788,  0.2185, -4.6841,  0.1343],
        [-0.1621, -1.7765,  0.1580,  2.6047, -0.7978]])
```

Lỗi A * B: The size of tensor a (4) must match the size of tensor b (5) at non-singleton dimension 1

A * b_row shape (3,1): torch.Size([3, 4])

A * b_col shape (4,) : torch.Size([3, 4])

In [6]: # Bài 4 (In-place vs Out-of-place).

```
# 1. So sánh giá trị tensor a sau a.add(b) và a.add_(b).
# 2. Khi nào nên/không nên dùng in-place (liên quan requires_grad).
```

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([10.0, 20.0, 30.0])

# Out-of-place: a không thay đổi
result = a.add(b)
print("Out-of-place a.add(b):", result)
print("a sau add()           :", a) # vẫn giữ nguyên [1, 2, 3]
```

```

a.add_(b)
print("\nIn-place a.add_(b):", a)    # [11, 22, 33]

# Khi nào KHÔNG nên dùng in-place?
# → Khi tensor có requires_grad=True vì PyTorch cần giữ giá trị gốc
# để tính gradient. In-place sẽ phá hỏng computation graph.
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
try:
    y.add_(1)    # có thể gây lỗi hoặc sai gradient
    print("\ny sau in-place trên requires_grad tensor:", y)
except RuntimeError as e:
    print("\nLỗi in-place với requires_grad:", e)

```

Out-of-place a.add(b): tensor([11., 22., 33.])
a sau add() : tensor([1., 2., 3.])

In-place a.add_(b): tensor([11., 22., 33.])

y sau in-place trên requires_grad tensor: tensor([3., 5.], grad_fn=<AddBackward0>)

```

In [7]: # Bài 5 (Broadcasting).
# 1. Gia'i thích cơ chế cộng torch.ones(3,4) với torch.tensor([1,2,3,4]).
# 2. Gia'i thích tại sao không cộng được với vector size 3 và cách sửa.

import torch

A = torch.ones(3, 4)
v4 = torch.tensor([1.0, 2.0, 3.0, 4.0])    # shape (4,)

# 1. Broadcasting: v4 (4,) → được mở rộng thành (1,4) → (3,4)
# Mỗi hàng của A cộng với v4.
print("ones(3,4) + [1,2,3,4]:\n", A + v4)

# 2. Vector size 3 không cộng được vì cột (4) ≠ 3
v3 = torch.tensor([1.0, 2.0, 3.0])    # shape (3,)
try:
    _ = A + v3
except RuntimeError as e:
    print("\nLỗi cộng vector size 3:", e)

# Cách sửa: reshape v3 thành (3,1) → broadcast theo cột
v3_col = v3.unsqueeze(1)    # shape (3, 1)
print("\nSửa bằng unsqueeze(1) → shape:", v3_col.shape)
print("ones(3,4) + v3.unsqueeze(1):\n", A + v3_col)

```



```
ones(3,4) + [1,2,3,4]:
tensor([[2., 3., 4., 5.],
        [2., 3., 4., 5.],
        [2., 3., 4., 5.]])
```

Lỗi cộng vector size 3: The size of tensor a (4) must match the size of tensor b (3) at non-singleton dimension 1

```
Sửa bằng unsqueeze(1) → shape: torch.Size([3, 1])
ones(3,4) + v3.unsqueeze(1):
tensor([[2., 2., 2., 2.],
        [3., 3., 3., 3.],
        [4., 4., 4., 4.]])
```

```
In [8]: # Bài 6. Reduction operations (dim).
# Cho ma trận 3x3. Tính và giả'i thích:
#   sum(), sum(dim=0), sum(dim=1), mean(dim=0),
#   max(dim=1), argmin(dim=0) và vai trò của keepdim=True.
import torch

M = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0],
                  [7.0, 8.0, 9.0]])

print("M:\n", M)

# Tổng toàn bộ
print("\nM.sum()          =", M.sum())          # 45

# sum(dim=0): cộng theo chiều hàng → kết quả shape (3,)
print("M.sum(dim=0)      =", M.sum(dim=0))      # [12, 15, 18]

# sum(dim=1): cộng theo chiều cột → kết quả shape (3,)
print("M.sum(dim=1)      =", M.sum(dim=1))      # [6, 15, 24]

# mean(dim=0): trung bình mỗi cột
print("M.mean(dim=0)     =", M.mean(dim=0))     # [4, 5, 6]

# max(dim=1): giá trị lớn nhất và index theo mỗi hàng
vals, idx = M.max(dim=1)
print("M.max(dim=1)  values:", vals, " indices:", idx)

# argmin(dim=0): index nhỏ nhất theo cada cột
print("M.argmax(dim=0) =", M.argmax(dim=0))    # [0, 0, 0]

# keepdim=True giữ nguyên số' chiều → quan trọng khi broadcast trong NN
print("\nM.sum(dim=0, keepdim=True):\n", M.sum(dim=0, keepdim=True)) # shape
# Ví dụ: chuẩn hóa từng cột (layer norm kiểu'u thu' công)
col_mean = M.mean(dim=0, keepdim=True) # (1,3) broadcast cùng M (3,3)
print("Chuẩn hóa trừ mean:\n", M - col_mean)
```

```

M:
  tensor([[1., 2., 3.],
          [4., 5., 6.],
          [7., 8., 9.]])

M.sum()          = tensor(45.)
M.sum(dim=0)     = tensor([12., 15., 18.])
M.sum(dim=1)     = tensor([ 6., 15., 24.])
M.mean(dim=0)    = tensor([4., 5., 6.])
M.max(dim=1)     values: tensor([3., 6., 9.]) indices: tensor([2, 2, 2])
M.argmax(dim=0)  = tensor([0, 0, 0])

M.sum(dim=0, keepdim=True):
  tensor([[12., 15., 18.]])
Chuẩn hóa trừ mean:
  tensor([[ -3., -3., -3.],
          [ 0., 0., 0.],
          [ 3., 3., 3.]])

```

```

In [9]: # Bài 7. Squeeze và Unsqueeze.
        # Tìm và sử dụng liên quan view, unsqueeze, squeeze.
        # Gia'i thích trường hợp cần thêm chiều batch cho ảnh.

import torch

# --- Đoạn code có lỗi gốc ---
# img = torch.randn(3, 64, 64)      # ảnh RGB 64x64
# batch = img.view(1, 3, 64, 64)    # Lỗi! nếu tensor không contiguous
# squeezed = batch.squeeze()        # shape (3,64,64) nếu dim=1 bị bỏ đúng
# out = squeezed.unsqueeze(dim=5)    # Lỗi! dim=5 vượt quá số chiều (3)

# --- Sử dụng lỗi ---
img = torch.randn(3, 64, 64)        # shape (3, 64, 64)
print("img shape:", img.shape)

# Cách đúng: dùng unsqueeze thay vì view để thêm chiều batch
batch = img.unsqueeze(0)             # shape (1, 3, 64, 64) – an toàn, luôn đúng
print("Sau unsqueeze(0):", batch.shape)

# squeeze() bỏ các chiều có kích thước = 1
squeezed = batch.squeeze(0)          # → (3, 64, 64)
print("Sau squeeze(0): ", squeezed.shape)

# unsqueeze dim hợp lệ (0..ndim)
out = squeezed.unsqueeze(0)          # thêm chiều batch lại → (1,3,64,64)
print("Sau unsqueeze(0) lại:", out.shape)

# Ý nghĩa thực tế:
# Mô hình CNN nhận đầu vào (batch, C, H, W).
# Khi inference ảnh đơn lẻ cần unsqueeze(0) để tạo pseudo-batch.

```

```

img shape: torch.Size([3, 64, 64])
Sau unsqueeze(0): torch.Size([1, 3, 64, 64])
Sau squeeze(0):   torch.Size([3, 64, 64])
Sau unsqueeze(0) lại: torch.Size([1, 3, 64, 64])

```

```

In [10]: # Bài 8. Chuyển đổi NumPy <-> Tensor và Shared Memory.
# Tìm hiểu from_numpy dùng chung bộ nhớ vs torch.tensor() tạo bản sao.

import torch
import numpy as np

arr = np.array([1.0, 2.0, 3.0])

# from_numpy → dùng chung bộ nhớ (shared memory)
t_shared = torch.from_numpy(arr)
print("Trước khi sửa arr:", arr, "| t_shared:", t_shared)

arr[0] = 99.0
print("Sau arr[0]=99:      ", arr, "| t_shared:", t_shared) # t_shared cũng t

# torch.tensor() → tạo bản sao độc lập
arr2 = np.array([1.0, 2.0, 3.0])
t_copy = torch.tensor(arr2)
arr2[0] = 99.0
print("\nArr2 sau sửa:", arr2, "| t_copy (bản sao):", t_copy) # t_copy không

# Tensor → NumPy (cũng shared nếu trên CPU)
t = torch.tensor([10.0, 20.0, 30.0])
np_from_t = t.numpy()
t[1] = 999.0
print("\nt sau sửa:", t, "| numpy:", np_from_t) # numpy cũng thay đổi

print("\nKết luận:")
print("- from_numpy() / .numpy() : dùng chung bộ nhớ → nhanh nhưng có side-effect")
print("- torch.tensor(np_arr)    : tạo bản sao → an toàn hơn")

```

Trước khi sửa arr: [1. 2. 3.] | t_shared: tensor([1., 2., 3.], dtype=torch.float64)

Sau arr[0]=99: [99. 2. 3.] | t_shared: tensor([99., 2., 3.], dtype=torch.float64)

Arr2 sau sửa: [99. 2. 3.] | t_copy (bản sao): tensor([1., 2., 3.], dtype=torch.float64)

t sau sửa: tensor([10., 999., 30.]) | numpy: [10. 999. 30.]

Kết luận:

- from_numpy() / .numpy() : dùng chung bộ nhớ → nhanh nhưng có side-effect
- torch.tensor(np_arr) : tạo bản sao → an toàn hơn

```

In [11]: # Bài 9. Chuyển tensor sang GPU.
# 1. Kiểm tra x.device trên máy cá nhân / Colab.
# 2. Gợi thích tại sao dữ liệu & mô hình phải ở cùng thiết bị.
# 3. Cách xử lý lỗi khi tensor có requires_grad=True hoặc GPU tensor → L

import torch

x = torch.tensor([1.0, 2.0, 3.0])
print("Device mặc định:", x.device) # cpu

# 1. Kiểm tra GPU

```

```

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print("Thiết bị sẽ dùng:", device)

x_gpu = x.to(device)
print("x trên", x_gpu.device)

# 2. Dữ liệu & mô hình phải cùng thiết bị:
#   PyTorch không tự động copy giữa CPU/GPU → RuntimeError nếu khác device
#   Ví dụ: model trên GPU, input trên CPU → lỗi.
print("\nGiải thích: phép tính chỉ thực hiện được khi cả hai tensor cùng dev

# 3. Chuyển tensor GPU (có requires_grad) sang NumPy
#   Phải: .detach() (bỏ grad) rồi .cpu() (về CPU) rồi .numpy()
t = torch.tensor([1.0, 2.0], requires_grad=True)
y = t * 2
try:
    _ = y.numpy() # lỗi: vãn trong computation graph
except RuntimeError as e:
    print("\nLỗi .numpy() khi requires_grad=True:", e)

# Cách đúng
np_arr = y.detach().numpy() # detach trước
print("Chuyển đúng (detach → numpy):", np_arr)

# Nếu tensor trên GPU: y.detach().cpu().numpy()

```

Device mặc định: cpu
Thiết bị sẽ dùng: cpu
x trên cpu

Giải thích: phép tính chỉ thực hiện được khi cả hai tensor cùng device.

Lỗi .numpy() khi requires_grad=True: Can't call numpy() on Tensor that requires grad. Use tensor.detach().numpy() instead.
Chuyển đúng (detach → numpy): [2. 4.]

```

In [12]: # Bài 10. Tính gradient tự động.
# Hàm  $y = 2x^2 + 5x$  tại  $x = 2.0$ .
# So sánh tính tay và dùng backward().
# Gia'i thích lỗi khi gọi backward() lần 2.

import torch

# Tính tay:  $dy/dx = 4x + 5 \rightarrow$  tại  $x=2$ :  $4*2 + 5 = 13$ 
print("Đạo hàm tay tại x=2: dy/dx = 4*2 + 5 =", 4*2 + 5)

x = torch.tensor(2.0, requires_grad=True)
y = 2 * x**2 + 5 * x
print("y tại x=2:", y.item())

y.backward()
print("Gradient (autograd):", x.grad.item()) # phải bằng 13

# Lỗi khi gọi backward() lần 2:
# PyTorch gia'i phóng computation graph sau lần backward đầu tiên.
try:

```

```

    y.backward()
except RuntimeError as e:
    print("\nLỗi backward() lần 2:", e)

# Cách giữ graph: backward(retain_graph=True)
x2 = torch.tensor(2.0, requires_grad=True)
y2 = 2 * x2**2 + 5 * x2
y2.backward(retain_graph=True)
print("\nLần 1 (retain_graph=True):", x2.grad.item())
# Lần 2 gradient sẽ tích lũy → cần zero_grad trước
x2.grad.zero_()
y2.backward()
print("Lần 2 (sau zero_grad):", x2.grad.item())

```

Đạo hàm tay tại $x=2$: $dy/dx = 4*2 + 5 = 13$

y tại $x=2$: 18.0

Gradient (autograd): 13.0

Lỗi backward() lần 2: Trying to backward through the graph a second time (or directly access saved tensors after they have already been freed). Saved intermediate values of the graph are freed when you call .backward() or autograd.grad(). Specify retain_graph=True if you need to backward through the graph a second time or if you need to access saved tensors after calling backward.

Lần 1 (retain_graph=True): 13.0

Lần 2 (sau zero_grad): 13.0

In [13]: *# Bài 11. Gradient tích lũy (accumulation bug).*
Phát hiện lỗi tích lũy gradient trong vòng lặp và khắc phục bằng zero_grad

```

import torch

x = torch.tensor(2.0, requires_grad=True)

print("=== Không zero_grad (lỗi tích lũy) ===")
for i in range(5):
    y = x ** 2          # dy/dx = 2x = 4
    y.backward()
    print(f" Epoch {i}: x.grad = {x.grad.item():.1f}") # 4, 8, 12, 16, 20

# Reset
x = torch.tensor(2.0, requires_grad=True)

print("\n=== Có zero_grad (đúng) ===")
for i in range(5):
    if x.grad is not None:
        x.grad.zero_() # hoặc optimizer.zero_grad() khi dùng optimizer
    y = x ** 2
    y.backward()
    print(f" Epoch {i}: x.grad = {x.grad.item():.1f}") # luôn = 4

```

=== Không zero_grad (lỗi tích lũy) ===

```
Epoch 0: x.grad = 4.0
Epoch 1: x.grad = 8.0
Epoch 2: x.grad = 12.0
Epoch 3: x.grad = 16.0
Epoch 4: x.grad = 20.0
```

=== Có zero_grad (đúng) ===

```
Epoch 0: x.grad = 4.0
Epoch 1: x.grad = 4.0
Epoch 2: x.grad = 4.0
Epoch 3: x.grad = 4.0
Epoch 4: x.grad = 4.0
```

```
In [14]: # Bài 12. detach() và torch.no_grad().
# Phân biệt hai phương pháp trong inference/validation.

import torch

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# --- detach() ---
# Tạo tensor mới từ x không có liên kết grad, nhưng vẫn chia sẻ data.
x_det = x.detach()
print("x.requires_grad      :", x.requires_grad)
print("x_det.requires_grad:", x_det.requires_grad) # False

# Computation với x_det không được track trong graph
z = x_det * 2
print("z.requires_grad (từ detach):", z.requires_grad) # False

# --- torch.no_grad() ---
# Context manager: tất cả tensor tạo trong block đều không track grad.
# Hiệu quả hơn detach khi inference toàn bộ model.
with torch.no_grad():
    z2 = x * 3
    print("\nz2.requires_grad (bên trong no_grad):", z2.requires_grad) # False

print("\nSo sánh:")
print("- detach(): tách 1 tensor khỏi graph, thường dùng để lấy giá trị ra ngoài.")
print("- no_grad(): tắt grad tracking cho toàn bộ block → tiết kiệm memory/tính toán.")
print("Inference/validation nên dùng no_grad() để tránh memory leak.")
```

```
x.requires_grad      : True
x_det.requires_grad: False
z.requires_grad (từ detach): False
```

```
z2.requires_grad (bên trong no_grad): False
```

So sánh:

- detach(): tách 1 tensor khỏi graph, thường dùng để lấy giá trị ra ngoài.
- no_grad(): tắt grad tracking cho toàn bộ block → tiết kiệm memory/tính toán.

Inference/validation nên dùng no_grad() để tránh memory leak.

```

In [15]: # Bài 13. Hiện thực Gradient Descent từ đầu.
# Tối ưu  $L(w) = (w-3)^2 + 2(w-3) + 1$ .
# Vẽ đồ thị loss và giá trị thích ảnh hưởng của  $lr = 0.1$  vs  $1.5$ .

import torch
import matplotlib.pyplot as plt

def loss_fn(w):
    return (w - 3)**2 + 2*(w - 3) + 1

def gradient_descent(lr, n_epochs=50):
    w = torch.tensor(0.0, requires_grad=True)
    losses = []
    for _ in range(n_epochs):
        L = loss_fn(w)
        losses.append(L.item())
        L.backward()
        with torch.no_grad():
            w -= lr * w.grad
        w.grad.zero_()
    return w.item(), losses

#  $L(w) = (w-3)^2 + 2(w-3) + 1 = (w-4)^2$  không phải vậy
# Đạo hàm:  $dL/dw = 2(w-3) + 2 = 0 \rightarrow w^* = 2$ 

w01, losses01 = gradient_descent(lr=0.1)
w15, losses15 = gradient_descent(lr=1.5)

print(f"lr=0.1 → w tối ưu = {w01:.4f}")
print(f"lr=1.5 → w tối ưu = {w15:.4f}")

plt.figure(figsize=(10, 4))
plt.plot(losses01, label='lr = 0.1 (hội tụ tốt)')
plt.plot(losses15, label='lr = 1.5 (dao động / phân kỳ)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Gradient Descent: ảnh hưởng của Learning Rate')
plt.legend()
plt.grid(True)
plt.show()

print("\nGiải thích:")
print("- lr=0.1: bước nhỏ, hội tụ ổn định về nghiệm tối ưu.")
print("- lr=1.5: bước lớn, w có thể vượt qua nghiệm và dao động hoặc phân kỳ")

lr=0.1 → w tối ưu = 2.0000
lr=1.5 → w tối ưu = -2251799545249792.0000

```

Giải thích:

- lr=0.1: bước nhỏ, hội tụ ổn định về nghiệm tối ưu.
- lr=1.5: bước lớn, w có thể vượt qua nghiệm và dao động hoặc phân kỳ.

/tmp/ipykernel_578971/3440025053.py:40: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()

```
In [16]: # Bài 14. Hàm Sigmoid và đạo hàm.
# Tự viết sigmoid, tính đạo hàm bằng autograd,
# so sánh với công thức toán. Gia'i thích vanishing gradient.

import torch
import matplotlib.pyplot as plt

# Định nghĩa sigmoid thủ công
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

# Đạo hàm toán học:  $\sigma'(x) = \sigma(x) * (1 - \sigma(x))$ 
def sigmoid_grad_math(x):
    s = sigmoid(x)
    return s * (1 - s)

# Tính đạo hàm bằng autograd
x = torch.tensor(0.0, requires_grad=True)
y = sigmoid(x)
y.backward()
print(f"Tại x=0: autograd grad = {x.grad.item():.6f}")
print(f"Tại x=0: công thức toán = {sigmoid_grad_math(torch.tensor(0.0)).item():.6f}")

# Vẽ sigmoid và đạo hàm
xs = torch.linspace(-10, 10, 200)
ys = sigmoid(xs)
dys = sigmoid_grad_math(xs)

plt.figure(figsize=(10, 4))
plt.plot(xs.numpy(), ys.detach().numpy(), label='sigmoid(x)')
plt.plot(xs.numpy(), dys.detach().numpy(), label="sigmoid'(x)")
plt.axhline(0, color='gray', linewidth=0.5)
plt.title('Sigmoid và Đạo Hàm')
plt.xlabel('x')
plt.grid(True)
plt.legend()
plt.show()

print("\nVanishing Gradient:")
print("Khi |x| lớn ( $\gg 0$  hoặc  $\ll 0$ ),  $\sigma'(x) \approx 0$ ." )
print("Trong mạng sâu, gradient nhân nhiều lần qua các lớp  $\rightarrow$  tiệm cận 0  $\rightarrow$  học rất chậm.")
print("Đây là lý do ReLU và các hàm khác được dùng thay sigmoid trong mạng s")
```

Tại x=0: autograd grad = 0.250000

Tại x=0: công thức toán = 0.250000

Vanishing Gradient:

Khi $|x|$ lớn ($\gg 0$ hoặc $\ll 0$), $\sigma'(x) \approx 0$.

Trong mạng sâu, gradient nhân nhiều lần qua các lớp $\rightarrow$ tiệm cận 0 $\rightarrow$ học rất chậm.

Đây là lý do ReLU và các hàm khác được dùng thay sigmoid trong mạng sâu.

/tmp/ipykernel_578971/3624952928.py:37: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()


```

In [17]: # Bài 15. Mini-project: Hồi Quy Tuyến Tính.
# Xây dựng mô hình  $\hat{y} = wx + b$  để dự báo dữ liệu từ  $y = 2.5X - 1.0 + \text{nhiều}$ .
# Train loop 200 epochs, MSE loss, cập nhật no_grad, vẽ kết quả.
# Gia'i thích lý do cần zero_grad().

import torch
import matplotlib.pyplot as plt

# ----- Sinh dữ liệu -----
torch.manual_seed(42)
X = torch.linspace(0, 10, 100).unsqueeze(1) # (100, 1)
noise = torch.randn_like(X) * 0.5
y_true = 2.5 * X - 1.0 + noise

# ----- Khởi tạo tham số -----
w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)
lr = 0.01
losses = []

# ----- Train loop -----
for epoch in range(200):
    # Forward
    y_pred = w * X + b

    # MSE Loss
    loss = ((y_pred - y_true) ** 2).mean()
    losses.append(loss.item())

    # Backward
    loss.backward()

    # Cập nhật tham số (không track grad)
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad

    # Xóa gradient sau mỗi bước (tránh tích lũy)
    w.grad.zero_()
    b.grad.zero_()

print(f"Kết quả học: w = {w.item():.4f}, b = {b.item():.4f}")
print(f"Giá trị thực: w = 2.5, b = -1.0")

# ----- Vẽ đồ thị Loss -----
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.plot(losses)
ax1.set_title('Loss theo Epoch')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('MSE Loss')
ax1.grid(True)

# ----- Vẽ dữ liệu và đường hồi quy -----
with torch.no_grad():

```

```

y_fit = w * X + b

ax2.scatter(X.numpy(), y_true.numpy(), s=10, alpha=0.5, label='Dữ liệu thực')
ax2.plot(X.numpy(), y_fit.numpy(), 'r', linewidth=2, label=f'Hồi quy: y={w}.i
ax2.set_title('Hồi Quy Tuyến Tính')
ax2.set_xlabel('X')
ax2.set_ylabel('y')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

print("\nLý do cần zero_grad():")
print("PyTorch tích lũy gradient qua các lần backward()")
print("→ phải gọi zero_grad() trước mỗi bước để gradient chỉ phản ánh batch

```

Kết quả học: $w = 2.4247$, $b = -0.4743$

Giá trị thực: $w = 2.5$, $b = -1.0$

Lý do cần zero_grad():

PyTorch tích lũy gradient qua các lần backward()

→ phải gọi zero_grad() trước mỗi bước để gradient chỉ phản ánh batch hiện tại.

/tmp/ipykernel_578971/2879118834.py:67: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
plt.show()